INGETIS / ITIS – Mastère 1

Application Fil Rouge QueenBee

Itinéraire d'urgence par l'algorithme de Dijkstra

Cas d'usage UC-C01 – Priorisation des visites et urgences

Binôme

Yaya MOHAMEDHEN

Djamel Eddine Ahmed BACHA

Spécialité

Développement / DevOps

Évaluation pratique d'algorithmique
Enseignante : Mme Maryem JRABA

Rendu : 5 novembre 2026

Sommaire

1 Contexte et modélisation **2**
1.1 Problème métier . . . 2
1.2 Modélisation en graphe . . . 2

2 Implémentation de l'algorithme **3**
2.1 File de priorité : tas binaire minimum . . . 3
2.2 Graphe, Dijkstra avec tas binaire et version naïve . . . 4
2.3 Données du site pilote . . . 7

3 Intégration dans l'API du backend QueenBee **8**

4 Jeux de tests et résultats **10**

5 Analyse de complexité **14**
5.1 Correction (rappel) . . . 14
5.2 Coût des opérations du tas . . . 15
5.3 Dijkstra avec tas binaire : $\mathcal{O}((|V| + |E|) \log |V|)$ . . . 15
5.4 Version naïve sans tas : $\mathcal{O}(|V|^2)$ . . . 15
5.5 Comparaison théorique . . . 15

6 Benchmark mesuré **16**

7 Conclusion **18**

A Exécution du projet **18**

Conformité à la règle impérative. Aucune bibliothèque de graphes (NetworkX, SciPy, Graph-tool…) ni module `heapq` n'est utilisé : la file de priorité (tas binaire) et l'algorithme sont entièrement implémentés à la main. FastAPI ne sert qu'à exposer l'endpoint REST, `pytest` qu'à exécuter les tests.

1 Contexte et modélisation

1.1 Problème métier

Lorsqu'une ruche déclenche une alerte critique (chute brutale de poids, risque d'essaimage, anomalie thermique), le coordinateur régional doit s'y rendre au plus vite (UC-C01, section 4.2 du cahier des charges). On cherche donc, sur le maillage des pistes du site, le chemin de *coût minimal* entre la position du coordinateur et la ruche à risque : c'est un problème de plus court chemin à source unique, avec des poids positifs. L'algorithme de Dijkstra est adapté, et il est correct précisément parce que les temps de trajet ne sont jamais négatifs.

1.2 Modélisation en graphe

- **Sommets** V : la base du coordinateur, les carrefours de pistes $(P_1, \dots, P_6)$ et les ruches, identifiées comme dans la trame capteur (section 6.5 : `SN-CASA-014`).
- **Arêtes** E : les tronçons de piste praticables, non orientés.
- **Poids** $w(u, v) \geq 0$: temps de trajet en minutes. Un poids négatif est refusé à la construction (exception).
- **Structure** : liste d'adjacence (`dict` → liste de couples (voisin, poids)), en mémoire $\mathcal{O}(|V| + |E|)$. Elle convient à un maillage de pistes, qui est un graphe *creux*.

La figure 1 présente le site pilote `SN-CASA` (Casamance, Sénégal). La ruche `SN-CASA-099`, coupée de toute piste pendant la saison des pluies, fournit le cas d'un graphe non connexe.

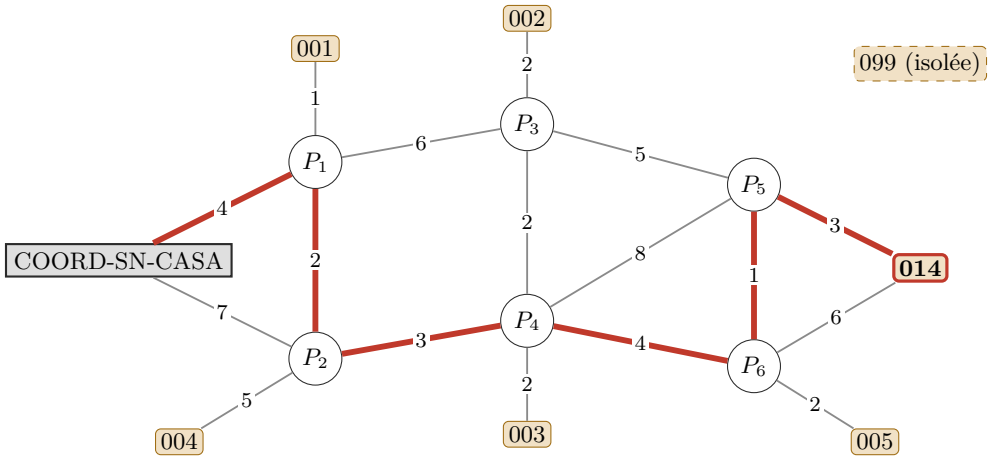


Figure 1 : Maillage des pistes du site SN-CASA (poids en minutes). En rouge, l'itinéraire optimal calculé vers la ruche en alerte `SN-CASA-014` : coût total 17 min.

On remarque que le chemin optimal passe par P_6 puis *revient* vers P_5 ($4 + 1 = 5$ min), alors que la liaison directe P_4–P_5 coûte 8 min. Un algorithme glouton naïf, qui prendrait toujours l'arête la plus courte, ne trouverait pas ce trajet.

2 Implémentation de l'algorithme

2.1 File de priorité : tas binaire minimum

Le tas est un arbre binaire presque complet stocké dans un tableau : le parent de l'indice i est $\lfloor (i-1)/2 \rfloor$, ses enfants sont $2i+1$ et $2i+2$. L'invariant de tas (chaque parent a une priorité inférieure ou égale à celle de ses enfants) garantit que le minimum se trouve à la racine. Comme la hauteur vaut $\lfloor \log_2 n \rfloor$, `push` et `pop` coûtent $\mathcal{O}(\log n)$.

Code 1: `app/min_heap.py` – tas binaire minimum

```
"""
min_heap.py -- File de priorite implementee a la main (tas binaire minimum).

Le tas est stocke dans un tableau Python :
  - le parent de l'indice i est (i - 1) // 2
  - ses enfants sont 2i + 1 (gauche) et 2i + 2 (droite)
Invariant : la priorite de chaque noeud est <= a celle de ses enfants,
donc la racine (indice 0) contient toujours le minimum.

Aucune bibliotheque externe (ni heapq) n'est utilisee.
"""

from typing import Any, List, Tuple


class MinHeap:
    """Tas binaire minimum de couples (priorite, valeur)."""

    def __init__(self) -> None:
        self._data: List[Tuple[float, Any]] = []

    def __len__(self) -> int:
        return len(self._data)

    def is_empty(self) -> bool:
        return len(self._data) == 0

    def push(self, priority: float, value: Any) -> None:
        """Insere un element en O(log n) : ajout en fin puis remontee."""
        self._data.append((priority, value))
        self._sift_up(len(self._data) - 1)

    def pop(self) -> Tuple[float, Any]:
        """Retire et renvoie l'element de plus petite priorite en O(log n)."""
        if self.is_empty():
            raise IndexError("pop sur un tas vide")
        root = self._data[0]
        last = self._data.pop()          # on retire le dernier element
        if self._data:                   # s'il reste des elements,
            self._data[0] = last         # il prend la place de la racine
            self._sift_down(0)           # puis redescend a sa place
        return root

    def peek(self) -> Tuple[float, Any]:
        """Consulte le minimum sans le retirer, en O(1)."""
        if self.is_empty():
```

```
            raise IndexError("peek sur un tas vide")
        return self._data[0]

    # ------------------------------------------------------------------
    # Operations internes
    # ------------------------------------------------------------------
    def _sift_up(self, i: int) -> None:
        """Fait remonter l'element i tant qu'il est plus petit que son
           parent.
        Au plus log2(n) echanges (hauteur de l'arbre)."""
        while i > 0:
            parent = (i - 1) // 2
            if self._data[i][0] < self._data[parent][0]:
                self._data[i], self._data[parent] = self._data[parent],
                   self._data[i]
                i = parent
            else:
                break

    def _sift_down(self, i: int) -> None:
        """Fait descendre l'element i vers le plus petit de ses enfants.
        Au plus log2(n) echanges."""
        n = len(self._data)
        while True:
            left, right = 2 * i + 1, 2 * i + 2
            smallest = i
            if left < n and self._data[left][0] <
               self._data[smallest][0]:
                smallest = left
            if right < n and self._data[right][0] <
               self._data[smallest][0]:
                smallest = right
            if smallest == i:
                break
            self._data[i], self._data[smallest] = self._data[smallest],
               self._data[i]
            i = smallest
```

2.2 Graphe, Dijkstra avec tas binaire et version naïve

Nous utilisons la variante *lazy deletion* : au lieu d'une opération `decrease-key`, qui obligerait à maintenir la position de chaque sommet dans le tas, on réinsère le sommet avec sa nouvelle distance, puis on ignore à l'extraction les entrées devenues obsolètes. La complexité asymptotique est la même (voir section 5), et le code reste plus simple. Le tableau `prev` mémorise le prédécesseur de chaque sommet et permet de reconstituer intégralement le chemin.

Code 2: `app/graph.py` – graphe, Dijkstra (tas binaire), version naïve, reconstitution du chemin

```
"""
graph.py -- Graphe pondere non oriente (liste d'adjacence) et
   algorithmes
de plus court chemin : Dijkstra avec tas binaire, et version naive
   O(V^2).

Aucune bibliotheque tierce de graphes n'est utilisee.
"""

import math
from typing import Dict, List, Optional, Tuple
```

```python

from app.min_heap import MinHeap

INF = math.inf


class Graph:
    """Graphe pondere stocke en liste d'adjacence.

    adj[u] = liste de couples (v, poids). Memoire : O(|V| + |E|).
    """

    def __init__(self, directed: bool = False) -> None:
        self.directed = directed
        self.adj: Dict[str, List[Tuple[str, float]]] = {}

    def add_node(self, node: str) -> None:
        if node not in self.adj:
            self.adj[node] = []

    def add_edge(self, u: str, v: str, weight: float) -> None:
        """Ajoute une arete u-v. Les poids negatifs sont refuses :
        Dijkstra n'est correct que pour des poids >= 0."""
        if weight < 0:
            raise ValueError(f"Poids negatif interdit ({u}-{v} :
                {weight})")
        self.add_node(u)
        self.add_node(v)
        self.adj[u].append((v, weight))
        if not self.directed:
            self.adj[v].append((u, weight))

    def has_node(self, node: str) -> bool:
        return node in self.adj

    def nodes(self) -> List[str]:
        return list(self.adj.keys())

    def edge_count(self) -> int:
        total = sum(len(neigh) for neigh in self.adj.values())
        return total if self.directed else total // 2


def reconstruct_path(prev: Dict[str, Optional[str]], source: str,
                     target: str) -> List[str]:
    """Remonte la chaine des predecesseurs de target jusqu'a source,
    puis inverse la liste. Cout O(longueur du chemin) <= O(|V|)."""
    path: List[str] = []
    node: Optional[str] = target
    while node is not None:
        path.append(node)
        if node == source:
            break
        node = prev.get(node)
    path.reverse()
    return path if path and path[0] == source else []


def dijkstra(graph: Graph, source: str,
             target: Optional[str] = None
```

```
69              ) -> Tuple[Dict[str, float], Dict[str, Optional[str]]]:
70      """Dijkstra a source unique avec tas binaire (lazy deletion).
71
72      Renvoie (dist, prev) :
73        dist[v] = cout minimal source -> v (INF si inatteignable)
74        prev[v] = predecesseur de v sur un plus court chemin
75      Si target est donne, on s'arrete des qu'il est extrait du tas
76      (sa distance est alors definitive).
77
78      Complexite : O((|V| + |E|) log |V|).
79      """
80      if not graph.has_node(source):
81          raise KeyError(f"Noeud de depart inconnu : {source}")
82
83      dist: Dict[str, float] = {v: INF for v in graph.adj}
84      prev: Dict[str, Optional[str]] = {v: None for v in graph.adj}
85      visited = set()
86
87      dist[source] = 0.0
88      heap = MinHeap()
89      heap.push(0.0, source)
90
91      while not heap.is_empty():
92          d_u, u = heap.pop()                  # O(log |V|)
93
94          # Lazy deletion : une ancienne entree (distance obsolete)
95          # peut rester dans le tas ; on l'ignore.
96          if u in visited:
97              continue
98          visited.add(u)                       # dist[u] est desormais
                definitive
99
100         if u == target:                      # arret anticipe
101             break
102
103         # Relachement des aretes sortantes de u
104         for v, w in graph.adj[u]:
105             if v in visited:
106                 continue
107             new_dist = d_u + w
108             if new_dist < dist[v]:
109                 dist[v] = new_dist
110                 prev[v] = u
111                 heap.push(new_dist, v)       # O(log |V|)
112
113     return dist, prev
114
115
116 def dijkstra_naive(graph: Graph, source: str
117                    ) -> Tuple[Dict[str, float], Dict[str,
                           Optional[str]]]:
118     """Version naive sans tas : a chaque iteration, on cherche par
119     parcours lineaire le sommet non visite de distance minimale.
120
121     Complexite : O(|V|^2 + |E|) = O(|V|^2).
122     """
123     if not graph.has_node(source):
124         raise KeyError(f"Noeud de depart inconnu : {source}")
125
126     dist: Dict[str, float] = {v: INF for v in graph.adj}
```

```
    prev: Dict[str, Optional[str]] = {v: None for v in graph.adj}
    visited = set()
    dist[source] = 0.0

    for _ in range(len(graph.adj)):           # |V| iterations
        u, best = None, INF
        for v in graph.adj:                   # parcours en O(|V|)
            if v not in visited and dist[v] < best:
                u, best = v, dist[v]
        if u is None:                         # le reste est
            inatteignable
            break
        visited.add(u)
        for v, w in graph.adj[u]:
            if v not in visited and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u

    return dist, prev


def shortest_path(graph: Graph, source: str,
                  target: str) -> Tuple[float, List[str]]:
    """Cout total et sequence ordonnee des noeuds de source a target.
    Renvoie (INF, []) si target est inatteignable."""
    if not graph.has_node(target):
        raise KeyError(f"Noeud cible inconnu : {target}")
    dist, prev = dijkstra(graph, source, target)
    if dist[target] == INF:
        return INF, []
    return dist[target], reconstruct_path(prev, source, target)
```

2.3 Données du site pilote

Code 3: `app/site_data.py` – maillage du site SN-CASA

```
"""
site_data.py -- Maillage des pistes et acces du site rucher SN-CASA
(Casamance, Senegal) utilise pour le cas d'usage UC-C01.

Sommets : base du coordinateur, carrefours de pistes (P1..P6), ruches.
Poids   : temps de trajet en minutes (piste, moto / 4x4).
La ruche SN-CASA-099 n'est reliee a aucune piste (acces coupe pendant
la saison des pluies) : elle sert de cas "non connexe".
"""

from app.graph import Graph

SITE_ID = "SN-CASA"

EDGES = [
    # Base du coordinateur et carrefours de pistes
    ("COORD-SN-CASA", "P1", 4), ("COORD-SN-CASA", "P2", 7),
    ("P1", "P2", 2), ("P1", "P3", 6), ("P2", "P4", 3),
    ("P3", "P4", 2), ("P3", "P5", 5), ("P4", "P5", 8),
    ("P4", "P6", 4), ("P5", "P6", 1),
    # Acces aux ruches
    ("P1", "SN-CASA-001", 1), ("P3", "SN-CASA-002", 2),
    ("P4", "SN-CASA-003", 2), ("P2", "SN-CASA-004", 5),
```

```
    ("P6", "SN-CASA-005", 2), ("P5", "SN-CASA-014", 3),
    ("P6", "SN-CASA-014", 6),
]

ISOLATED_NODES = ["SN-CASA-099"]


def build_site_graph() -> Graph:
    g = Graph(directed=False)
    for u, v, w in EDGES:
        g.add_edge(u, v, w)
    for node in ISOLATED_NODES:
        g.add_node(node)
    return g


# Registre des graphes par site (un seul site pilote ici)
SITE_GRAPHS = {SITE_ID: build_site_graph()}
```

3 Intégration dans l'API du backend QueenBee

L'algorithme n'est pas un script isolé : il est exposé par un endpoint REST du backend (FastAPI, cohérent avec la stack recommandée en section 9.1 et avec les endpoints de la section 10.1). L'identifiant de la ruche détermine le site, donc le graphe à utiliser (`SN-CASA-014` → `SN-CASA`). Le temps de calcul est mesuré avec `time.perf_counter()` autour du seul appel algorithmique.

Code 4: `app/main.py` – endpoint `POST /api/v1/routing/emergency-path`

```
"""
main.py -- Endpoint REST du backend QueenBee (FastAPI) exposant le
    calcul
d'itineraire d'urgence (UC-C01).

POST /api/v1/routing/emergency-path
  Entree  : start_id (coordinateur / point de depart), hive_id (ruche
     cible)
  Sortie  : cout total, sequence des points de passage, temps de calcul
"""

import time
from typing import List

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

from app.graph import INF, shortest_path
from app.site_data import SITE_GRAPHS

app = FastAPI(title="QueenBee - Routing API", version="1.0.0")


class EmergencyPathRequest(BaseModel):
    start_id: str = Field(..., min_length=1, examples=["COORD-SN-CASA"])
    hive_id: str = Field(..., min_length=1, examples=["SN-CASA-014"])


class EmergencyPathResponse(BaseModel):
    site_id: str
    start_id: str
```

```
    hive_id: str
    total_cost: float
    unit: str = "min"
    path: List[str]
    hops: int
    compute_time_ms: float


def site_of(hive_id: str) -> str:
    """Le site est le prefixe de l'identifiant de ruche : SN-CASA-014
       -> SN-CASA."""
    return "-".join(hive_id.split("-")[:2])


@app.get("/api/v1/health")
def health() -> dict:
    return {"status": "ok"}


@app.post("/api/v1/routing/emergency-path",
   response_model=EmergencyPathResponse)
def emergency_path(req: EmergencyPathRequest) -> EmergencyPathResponse:
    site_id = site_of(req.hive_id)
    graph = SITE_GRAPHS.get(site_id)
    if graph is None:
        raise HTTPException(404, f"Site inconnu : {site_id}")
    if not graph.has_node(req.start_id):
        raise HTTPException(404, f"Point de depart inconnu :
           {req.start_id}")
    if not graph.has_node(req.hive_id):
        raise HTTPException(404, f"Ruche inconnue : {req.hive_id}")

    t0 = time.perf_counter()
    cost, path = shortest_path(graph, req.start_id, req.hive_id)
    elapsed_ms = (time.perf_counter() - t0) * 1000

    if cost == INF:
        raise HTTPException(404, f"Aucun itineraire vers {req.hive_id} "
                                 f"(ruche inaccessible depuis
                                    {req.start_id})")

    return EmergencyPathResponse(
        site_id=site_id, start_id=req.start_id, hive_id=req.hive_id,
        total_cost=cost, path=path, hops=len(path) - 1,
        compute_time_ms=round(elapsed_ms, 4),
    )
```

Exemple d'appel réel

Code 5: Requête

```
POST /api/v1/routing/emergency-path
Content-Type: application/json

{ "start_id": "COORD-SN-CASA", "hive_id": "SN-CASA-014" }
```

Code 6: Réponse `200 OK`

```
{
```

```
  "site_id": "SN-CASA",
  "start_id": "COORD-SN-CASA",
  "hive_id": "SN-CASA-014",
  "total_cost": 17.0,
  "unit": "min",
  "path": ["COORD-SN-CASA", "P1", "P2", "P4", "P6", "P5",
     "SN-CASA-014"],
  "hops": 6,
  "compute_time_ms": 0.0684
}
```

Situation	Code HTTP	Réponse
Itinéraire trouvé	200	coût, chemin, temps de calcul
Ruche inaccessible (non connexe)	404	« Aucun itinéraire vers ... »
Ruche, point de départ ou site inconnu	404	message explicite
Corps de requête invalide	422	validation Pydantic

Tableau 1: Gestion des cas d'erreur par l'endpoint

4 Jeux de tests et résultats

Les tests couvrent quatre niveaux : le tas seul, l'algorithme sur des cas normaux et des cas limites, la cohérence entre la version avec tas et la version naïve (20 graphes aléatoires), et l'endpoint REST via le client de test FastAPI.

Cas testé	Résultat attendu
Tas : insertions puis extractions	ordre croissant (y compris 1 000 valeurs aléatoires)
Tas vide	exception `IndexError`
Site SN-CASA, base → 014	coût 17, chemin exact de la figure 1
Détour plus court que l'arête directe	le détour est choisi
Départ = arrivée	coût 0, chemin $[s]$
Graphe non connexe (ruche 099)	coût ∞, chemin vide
Graphe avec cycles	distances exactes, pas de boucle infinie
Arêtes de poids nul	coût 0
Arêtes multiples entre deux sommets	la moins coûteuse est retenue
Deux chemins de même coût	un chemin optimal valide
Poids négatif	refusé (`ValueError`)
Sommet inconnu	exception `KeyError`
Graphe orienté	chemin dans un seul sens
Tas vs naïf, 20 graphes aléatoires	distances identiques
API : 200 / 404 (inaccessible, inconnue, site) / 422	codes et contenus attendus

Tableau 2: Synthèse des cas de test

Code 7: `tests/test_dijkstra.py` – tests unitaires (pytest)

```
"""Tests unitaires : tas binaire, Dijkstra, version naive et endpoint
    REST."""

import random

import pytest
from fastapi.testclient import TestClient

```

```python
from app.graph import INF, Graph, dijkstra, dijkstra_naive, shortest_path
from app.main import app
from app.min_heap import MinHeap
from app.site_data import build_site_graph

client = TestClient(app)


# ---------------------------------------------------------------- Tas binaire
def test_heap_pop_renvoie_ordre_croissant():
    h = MinHeap()
    values = [7, 3, 9, 1, 4, 1, 8, 0, 6]
    for x in values:
        h.push(x, f"n{x}")
    out = [h.pop()[0] for _ in range(len(values))]
    assert out == sorted(values)


def test_heap_vide_leve_une_erreur():
    with pytest.raises(IndexError):
        MinHeap().pop()


def test_heap_aleatoire_1000_elements():
    h = MinHeap()
    values = [random.random() for _ in range(1000)]
    for x in values:
        h.push(x, None)
    assert [h.pop()[0] for _ in range(1000)] == sorted(values)


# ---------------------------------------------------------------- Dijkstra : normal
def test_chemin_site_sn_casa():
    g = build_site_graph()
    cost, path = shortest_path(g, "COORD-SN-CASA", "SN-CASA-014")
    assert cost == 17
    assert path == ["COORD-SN-CASA", "P1", "P2", "P4", "P6", "P5", "SN-CASA-014"]


def test_le_detour_peut_etre_plus_court():
    # A-C direct coute 10, mais A-B-C coute 3 : le chemin indirect gagne
    g = Graph()
    g.add_edge("A", "C", 10)
    g.add_edge("A", "B", 1)
    g.add_edge("B", "C", 2)
    assert shortest_path(g, "A", "C") == (3, ["A", "B", "C"])


# ---------------------------------------------------------------- Dijkstra : cas limites
def test_depart_egal_arrivee():
    g = build_site_graph()
    assert shortest_path(g, "P3", "P3") == (0, ["P3"])


def test_graphe_non_connexe():
```

```
63      g = build_site_graph()
64      cost, path = shortest_path(g, "COORD-SN-CASA", "SN-CASA-099")
65      assert cost == INF and path == []
66
67
68  def test_graphe_avec_cycles():
69      # Triangle A-B-C + boucle C-D-A : l'algorithme ne boucle pas
70      g = Graph()
71      for u, v, w in [("A", "B", 1), ("B", "C", 1), ("C", "A", 5),
72                      ("C", "D", 1), ("D", "A", 4)]:
73          g.add_edge(u, v, w)
74      dist, _ = dijkstra(g, "A")
75      assert dist == {"A": 0, "B": 1, "C": 2, "D": 3}
76
77
78  def test_arete_de_poids_nul():
79      g = Graph()
80      g.add_edge("A", "B", 0)
81      g.add_edge("B", "C", 0)
82      assert shortest_path(g, "A", "C") == (0, ["A", "B", "C"])
83
84
85  def test_aretes_multiples_entre_deux_noeuds():
86      g = Graph()
87      g.add_edge("A", "B", 9)
88      g.add_edge("A", "B", 2)        # deux pistes differentes
89      assert shortest_path(g, "A", "B")[0] == 2
90
91
92  def test_deux_chemins_de_meme_cout():
93      g = Graph()
94      g.add_edge("A", "B", 1); g.add_edge("B", "D", 1)
95      g.add_edge("A", "C", 1); g.add_edge("C", "D", 1)
96      cost, path = shortest_path(g, "A", "D")
97      assert cost == 2 and path[0] == "A" and path[-1] == "D" and
            len(path) == 3
98
99
100 def test_poids_negatif_refuse():
101     with pytest.raises(ValueError):
102         Graph().add_edge("A", "B", -1)
103
104
105 def test_noeud_inconnu():
106     g = build_site_graph()
107     with pytest.raises(KeyError):
108         shortest_path(g, "INCONNU", "SN-CASA-014")
109     with pytest.raises(KeyError):
110         shortest_path(g, "COORD-SN-CASA", "INCONNU")
111
112
113 def test_graphe_oriente():
114     g = Graph(directed=True)
115     g.add_edge("A", "B", 1)
116     assert shortest_path(g, "A", "B")[0] == 1
117     assert shortest_path(g, "B", "A") == (INF, [])
118
119
120 # ---------------------------------------- Coherence tas binaire /
        version naive
```

```python
@pytest.mark.parametrize("seed", range(20))
def test_tas_et_naif_donnent_les_memes_distances(seed):
    rng = random.Random(seed)
    g = Graph()
    n = 60
    for i in range(n):
        g.add_node(str(i))
    for _ in range(150):
        u, v = rng.randrange(n), rng.randrange(n)
        g.add_edge(str(u), str(v), rng.randint(0, 20))
    d_heap, _ = dijkstra(g, "0")
    d_naive, _ = dijkstra_naive(g, "0")
    assert d_heap == d_naive


# ---------------------------------------------------------------- Endpoint REST
def test_api_chemin_ok():
    r = client.post("/api/v1/routing/emergency-path",
                    json={"start_id": "COORD-SN-CASA", "hive_id": "SN-CASA-014"})
    assert r.status_code == 200
    body = r.json()
    assert body["total_cost"] == 17
    assert body["path"][0] == "COORD-SN-CASA" and body["path"][-1] == "SN-CASA-014"
    assert body["hops"] == 6
    assert body["compute_time_ms"] >= 0


def test_api_ruche_inaccessible():
    r = client.post("/api/v1/routing/emergency-path",
                    json={"start_id": "COORD-SN-CASA", "hive_id": "SN-CASA-099"})
    assert r.status_code == 404


def test_api_ruche_inconnue():
    r = client.post("/api/v1/routing/emergency-path",
                    json={"start_id": "COORD-SN-CASA", "hive_id": "SN-CASA-777"})
    assert r.status_code == 404


def test_api_site_inconnu():
    r = client.post("/api/v1/routing/emergency-path",
                    json={"start_id": "COORD-SN-CASA", "hive_id": "CM-OUEST-001"})
    assert r.status_code == 404


def test_api_entree_invalide():
    r = client.post("/api/v1/routing/emergency-path", json={"start_id": ""})
    assert r.status_code == 422
```

Résultat de l'exécution

Code 8: `python -m pytest -v` – 39 tests, 39 réussis

```
$ python -m pytest -v
```

```
test_heap_pop_renvoie_ordre_croissant PASSED     [  2%]
test_heap_vide_leve_une_erreur PASSED            [  5%]
test_heap_aleatoire_1000_elements PASSED         [  7%]
test_chemin_site_sn_casa PASSED                  [ 10%]
test_le_detour_peut_etre_plus_court PASSED       [ 12%]
test_depart_egal_arrivee PASSED                  [ 15%]
test_graphe_non_connexe PASSED                   [ 17%]
test_graphe_avec_cycles PASSED                   [ 20%]
test_arete_de_poids_nul PASSED                   [ 23%]
test_aretes_multiples_entre_deux_noeuds PASSED   [ 25%]
test_deux_chemins_de_meme_cout PASSED            [ 28%]
test_poids_negatif_refuse PASSED                 [ 30%]
test_noeud_inconnu PASSED                        [ 33%]
test_graphe_oriente PASSED                       [ 35%]
test_tas_et_naif_donnent_les_memes_distances[0] PASSED [ 38%]
test_tas_et_naif_donnent_les_memes_distances[1] PASSED [ 41%]
test_tas_et_naif_donnent_les_memes_distances[2] PASSED [ 43%]
test_tas_et_naif_donnent_les_memes_distances[3] PASSED [ 46%]
test_tas_et_naif_donnent_les_memes_distances[4] PASSED [ 48%]
test_tas_et_naif_donnent_les_memes_distances[5] PASSED [ 51%]
test_tas_et_naif_donnent_les_memes_distances[6] PASSED [ 53%]
test_tas_et_naif_donnent_les_memes_distances[7] PASSED [ 56%]
test_tas_et_naif_donnent_les_memes_distances[8] PASSED [ 58%]
test_tas_et_naif_donnent_les_memes_distances[9] PASSED [ 61%]
test_tas_et_naif_donnent_les_memes_distances[10] PASSED [ 64%]
test_tas_et_naif_donnent_les_memes_distances[11] PASSED [ 66%]
test_tas_et_naif_donnent_les_memes_distances[12] PASSED [ 69%]
test_tas_et_naif_donnent_les_memes_distances[13] PASSED [ 71%]
test_tas_et_naif_donnent_les_memes_distances[14] PASSED [ 74%]
test_tas_et_naif_donnent_les_memes_distances[15] PASSED [ 76%]
test_tas_et_naif_donnent_les_memes_distances[16] PASSED [ 79%]
test_tas_et_naif_donnent_les_memes_distances[17] PASSED [ 82%]
test_tas_et_naif_donnent_les_memes_distances[18] PASSED [ 84%]
test_tas_et_naif_donnent_les_memes_distances[19] PASSED [ 87%]
test_api_chemin_ok PASSED                        [ 89%]
test_api_ruche_inaccessible PASSED               [ 92%]
test_api_ruche_inconnue PASSED                   [ 94%]
test_api_site_inconnu PASSED                     [ 97%]
test_api_entree_invalide PASSED                  [100%]
```

Bilan : 39 tests passés, 0 échec (le test de cohérence est paramétré sur 20 graines aléatoires).

5 Analyse de complexité

Notations : $n = |V|$ sommets, $m = |E|$ arêtes. Pour un graphe non orienté, la liste d'adjacence contient $2m$ arcs.

5.1 Correction (rappel)

Lemme 1 (Invariant de Dijkstra)**.** *Lorsqu'un sommet u est extrait du tas pour la première fois (et ajouté à `visited`),* $\mathrm{dist}[u]$ *est égal au coût du plus court chemin $\delta(s, u)$.*

Démonstration. Par l'absurde, soit u le premier sommet extrait avec $\mathrm{dist}[u] > \delta(s, u)$. Sur un plus court chemin de s à u, soit y le premier sommet non visité et x son prédécesseur, déjà visité. Comme x a été traité correctement, l'arête (x, y) a été relâchée, donc $\mathrm{dist}[y] = \delta(s, y)$. Les poids étant positifs ou nuls, $\delta(s, y) \leq \delta(s, u)$. On obtient $\mathrm{dist}[y] = \delta(s, y) \leq \delta(s, u) < \mathrm{dist}[u]$: le tas aurait dû extraire y avant u, ce qui est une contradiction. □

Cette preuve repose sur l'hypothèse $w \geq 0$. C'est pourquoi le constructeur du graphe refuse les poids négatifs.

5.2 Coût des opérations du tas

Lemme 2. *Dans un tas binaire de k éléments, `push` et `pop` coûtent $\mathcal{O}(\log k)$, et `peek` coûte $\mathcal{O}(1)$.*

Démonstration. L'arbre est presque complet, donc sa hauteur vaut $h = \lfloor \log_2 k \rfloor$. `_sift_up` remonte d'un niveau par échange, et `_sift_down` descend d'un niveau par échange : chacune effectue au plus h échanges en temps constant, soit $\mathcal{O}(\log k)$. □

5.3 Dijkstra avec tas binaire : $\mathcal{O}((|V| + |E|) \log |V|)$

Théorème 1. *La fonction `dijkstra` s'exécute en $\mathcal{O}((n + m) \log n)$ en temps et en $\mathcal{O}(n + m)$ en mémoire.*

Démonstration. On décompte chaque partie de l'algorithme.

1. **Initialisation** des tableaux `dist` et `prev` : $\mathcal{O}(n)$.
2. **Nombre d'insertions.** Un `push` n'a lieu qu'après un relâchement réussi. Chaque arc (u, v) n'est examiné qu'une fois, au moment où u est visité (grâce au test `if u in visited`). Il y a donc au plus $1 + 2m$ insertions, soit $\mathcal{O}(m)$.
3. **Nombre d'extractions.** On ne peut pas extraire plus d'éléments qu'on n'en a inséré : au plus $1 + 2m$ extractions. Parmi elles, au plus n sont valides ; les autres sont des entrées obsolètes, ignorées en $\mathcal{O}(1)$ après le `pop`.
4. **Taille du tas.** Elle reste bornée par $k \leq 1 + 2m$. Chaque opération coûte donc $\mathcal{O}(\log m)$. Or $m \leq n^2$, d'où $\log m \leq 2 \log n$ et $\mathcal{O}(\log m) = \mathcal{O}(\log n)$.
5. **Relâchements.** Le parcours de toutes les listes d'adjacence coûte $\mathcal{O}(n + m)$, hors opérations sur le tas.

Au total : $\mathcal{O}(n) + \mathcal{O}(m) \cdot \mathcal{O}(\log n) + \mathcal{O}(n + m) = \mathcal{O}((n + m) \log n)$. Lorsque le graphe est connexe ($m \geq n - 1$), cela donne $\mathcal{O}(m \log n)$. En mémoire, on stocke la liste d'adjacence ($\mathcal{O}(n + m)$), les tableaux `dist`, `prev` et `visited` ($\mathcal{O}(n)$), et le tas ($\mathcal{O}(m)$). La reconstitution du chemin coûte $\mathcal{O}(n)$, puisqu'un chemin simple compte au plus n sommets. □

Remarque. Avec `decrease-key`, le tas ne contiendrait jamais plus de n éléments, mais la borne reste $\mathcal{O}((n + m) \log n)$. La lazy deletion change seulement la taille maximale du tas ($\mathcal{O}(m)$ au lieu de $\mathcal{O}(n)$), sans effet sur la complexité asymptotique. Un tas de Fibonacci atteindrait $\mathcal{O}(m + n \log n)$, au prix d'une implémentation bien plus lourde et de constantes élevées.

5.4 Version naïve sans tas : $\mathcal{O}(|V|^2)$

Théorème 2. *La fonction `dijkstra_naive` s'exécute en $\mathcal{O}(n^2)$.*

Démonstration. La boucle externe effectue au plus n itérations. À chacune, la recherche du sommet non visité de distance minimale parcourt les n sommets, soit $\mathcal{O}(n)$. Les relâchements représentent $2m$ examens d'arcs au total, en $\mathcal{O}(1)$ chacun. Le coût total est donc $\mathcal{O}(n^2 + m)$, et comme $m \leq n^2$, il vaut $\mathcal{O}(n^2)$. □

5.5 Comparaison théorique

La version avec tas est préférable lorsque $m \log n < n^2$, c'est-à-dire $m = o(n^2 / \log n)$:

- **Graphe creux**, $m = \Theta(n)$: $\mathcal{O}(n \log n)$ contre $\mathcal{O}(n^2)$. Le tas l'emporte largement, et l'écart grandit comme $n / \log n$.
- **Graphe dense**, $m = \Theta(n^2)$: $\mathcal{O}(n^2 \log n)$ contre $\mathcal{O}(n^2)$. Asymptotiquement, la version naïve devient même légèrement meilleure.

Un maillage de pistes est un graphe creux (degré moyen d'un carrefour compris entre 2 et 4) : le tas binaire est donc le bon choix pour QueenBee.

6 Benchmark mesuré

Protocole. Graphes aléatoires connexes (un arbre couvrant aléatoire, complété par des arêtes aléatoires), poids entiers dans $[1, 100]$, calcul depuis le sommet 0 vers tous les autres. Pour chaque taille, on retient le meilleur temps sur 3 exécutions. Deux familles : *creux* ($m = 3n$) et *dense* ($m = n^2/4$, densité d'environ 50 %). Environnement : Python 3.13, Linux.

Type	$\lvert V\rvert$	$\lvert E\rvert$	Tas (ms)	Naïf (ms)	Gain (naïf / tas)
Creux	250	750	0,77	2,12	×2,8
Creux	500	1 500	1,72	8,65	×5,0
Creux	1 000	3 000	4,34	39,71	×9,2
Creux	2 000	6 000	12,37	161,50	×13,1
Creux	4 000	12 000	21,74	663,90	×30,5
Creux	8 000	24 000	56,52	3 142,25	**×55,6**
Dense	100	2 500	0,80	0,67	×0,84
Dense	200	10 000	3,29	3,18	×0,97
Dense	400	40 000	13,57	14,15	×1,04
Dense	800	160 000	67,06	79,00	×1,18
Dense	1 600	640 000	422,54	408,16	×0,97

Tableau 3: Temps d'exécution mesurés (`bench/benchmark.py`)

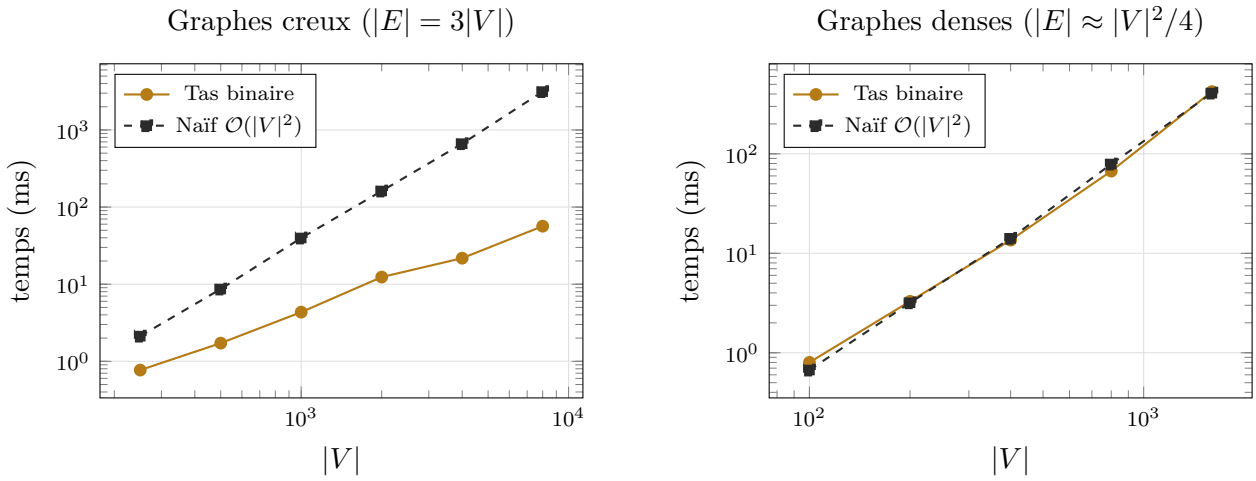


Figure 2: Temps d'exécution en échelle log–log

Interprétation

- **Graphes creux.** Sur l'échelle log–log, la pente de la version naïve vaut environ 2 : quand $|V|$ double, le temps est multiplié par 4 à 5, ce qui confirme $\mathcal{O}(|V|^2)$. La pente de la version avec tas est proche de 1, ce qui correspond à $\mathcal{O}(|V| \log |V|)$. Le gain passe de ×2,8 à ×55,6 et suit bien la croissance théorique en $|V|/\log |V|$.
- **Graphes denses.** Les deux versions sont à égalité (gain compris entre ×0,84 et ×1,18). C'est conforme à la théorie : avec $|E| = \Theta(|V|^2)$, le relâchement des arêtes domine les deux algorithmes, et le facteur $\log |V|$ du tas est compensé en pratique, car seule une partie des relâchements aboutit à un `push`.
- **Pour QueenBee.** Un site compte une cinquantaine de ruches et quelques dizaines de carrefours, sur un graphe creux. Le calcul prend environ 0,07 ms (mesuré par l'API), ce qui est

négligeable devant l'objectif de 500 ms (p95) fixé pour l'API centrale (section 8.1). La version avec tas reste la plus robuste si le maillage grandit, par exemple pour un graphe agrégé de plusieurs sites.

Code 9: `bench/benchmark.py` – script de mesure

```
"""
benchmark.py -- Mesure comparee : Dijkstra avec tas binaire vs version naive.

Deux familles de graphes aleatoires connexes, poids entiers dans [1, 100] :
  - creux : |E| ~ 3|V| (realiste pour un maillage de pistes)
  - dense : |E| ~ |V|^2 / 4 (densite 50 %)
Pour chaque taille, on garde le meilleur temps sur 3 executions.
Resultats ecrits dans bench/results.csv.
"""

import csv
import random
import sys
import time
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from app.graph import Graph, dijkstra, dijkstra_naive  # noqa: E402


def random_graph(n: int, m: int, seed: int) -> Graph:
    rng = random.Random(seed)
    g = Graph()
    for i in range(1, n):                        # arbre couvrant -> connexe
        g.add_edge(str(i), str(rng.randrange(i)), rng.randint(1, 100))
    existing = n - 1
    while existing < m:
        u, v = rng.randrange(n), rng.randrange(n)
        if u != v:
            g.add_edge(str(u), str(v), rng.randint(1, 100))
            existing += 1
    return g


def best_time(fn, g: Graph, repeat: int = 3) -> float:
    best = float("inf")
    for _ in range(repeat):
        t0 = time.perf_counter()
        fn(g, "0")
        best = min(best, time.perf_counter() - t0)
    return best * 1000  # ms


def main() -> None:
    rows = []
    for kind, sizes, edges in [
        ("creux", [250, 500, 1000, 2000, 4000, 8000], lambda n: 3 * n),
        ("dense", [100, 200, 400, 800, 1600], lambda n: n * n // 4),
    ]:
        for n in sizes:
            g = random_graph(n, edges(n), seed=n)
```

```
            t_heap = best_time(dijkstra, g)
            t_naive = best_time(dijkstra_naive, g)
            rows.append({"type": kind, "V": n, "E": g.edge_count(),
                         "heap_ms": round(t_heap, 2), "naive_ms":
                            round(t_naive, 2),
                         "ratio": round(t_naive / t_heap, 2)})
            print(rows[-1], flush=True)

    out = Path(__file__).parent / "results.csv"
    with out.open("w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=rows[0].keys())
        w.writeheader()
        w.writerows(rows)


if __name__ == "__main__":
    main()
```

7 Conclusion

Nous avons implémenté Dijkstra avec un tas binaire codé à la main, reconstitué intégralement le chemin optimal et exposé le calcul par l'endpoint REST `POST /api/v1/routing/emergency-path` du backend QueenBee. Les 39 tests valident les cas normaux et les cas limites (graphe non connexe, cycles, poids nuls, sommets inconnus, erreurs d'API). L'analyse théorique, en $\mathcal{O}((|V|+|E|)\log|V|)$ contre $\mathcal{O}(|V|^2)$, est confirmée par les mesures : le tas est jusqu'à 55 fois plus rapide sur un graphe creux comme un maillage de pistes, et équivalent sur un graphe dense.

Perspectives. Utiliser des poids dynamiques (état des pistes en saison des pluies, données météo), calculer un ordre de visite lorsque plusieurs alertes sont simultanées, et mettre en cache l'arbre des plus courts chemins depuis chaque base de coordinateur.

A Exécution du projet

```
pip install fastapi uvicorn pytest httpx
python -m pytest -v                    # 39 tests
python bench/benchmark.py              # benchmark -> bench/results.csv
uvicorn app.main:app --reload          # API sur
    http://127.0.0.1:8000/docs
```